

Smart Serial Controller: UART, SPI and I2C on the ZedBoard

This is Project 10, "From Protocol Converter to Smart Serial Controller". It contains the complete Verilog design, three self-checking testbenches, Vivado build scripts for the ZedBoard, and the bare-metal C program for the Zynq ARM.

New here? Start with [GETTING_STARTED.md](#). It is a step-by-step checklist: download, install, simulate, Way 1, Way 2. On Windows you do not need to type any commands. Double-click the scripts in `windows\`:

- `1_simulate.bat`
- `2_build_way1_fpga_only.bat`
- `3_build_way2_arm_fpga.bat`
- `4_open_project_gui.bat`

This README is the full reference.

The design follows the architecture in the slides:

```
Zynq PS (ARM) --AXI--> AXI->APB bridge --APB--> register bank --> UART block --> JA PmodUSBUART
                        |
                        | --> SPI block --> JB PmodSF3
                        |
                        | --> I2C block --> JC PmodTMP2
                        |
                        | bridge engine (FIFO -> FIFO, no CPU)
IRQ <-- interrupt controller --> JD logic analyser
```

Feature on the slides	Where it is	Proven by
APB register bank driven by the ARM	<code>rtl/ssc_regbank.v</code> , <code>rtl/ssc_axi_apb_bridge.v</code>	tb test 1, <code>tb_ssc_axi</code>
16-entry TX and RX FIFOs in every block	<code>rtl/ssc_fifo.v</code>	tests 6, 22
UART 5-8 bits, parity, 1 or 2 stop bits	<code>rtl/ssc_uart.v</code>	tests 2-4, 10
Fractional baud divider (54.25 → 0.006 %)	<code>rtl/ssc_baud_gen.v</code>	test 9 (all standard rates)
16× sampling with a vote on the middle 3 samples, glitch filter	<code>ssc_uart.v</code> , <code>ssc_sync_filter.v</code>	test 8
Parity, framing and break errors, overflow, RTS/CTS	<code>ssc_uart.v</code>	tests 5-7
SPI modes 0-3, 4-32-bit words, MSB/LSB first, 4 chip selects	<code>rtl/ssc_spi.v</code>	tests 11-13, 16
SPI master and slave, clean abort	<code>ssc_spi.v</code>	tests 14, 15
I2C 7- and 10-bit addresses, 100 and 400 kHz, repeated START	<code>rtl/ssc_i2c.v</code>	tests 17-20
Clock stretching, arbitration, NACK, bus busy	<code>ssc_i2c.v</code>	tests 19, 21, 23, 24
Bridge engine: two-way, pass-through (the paper's MPCU idea)	<code>rtl/ssc_bridge.v</code>	tests 25-27
Interrupt controller, one IRQ line	<code>rtl/ssc_irq.v</code>	test 28, <code>tb_ssc_axi</code>
Way 1: FPGA-only bring-up (type <code>t</code> → temperature)	<code>rtl/ssc_cmd_fsm.v</code> , <code>boards/zedboard/zed_top_standalone.v</code>	<code>tb_zed_standalone</code>
Way 2: ARM + FPGA with a C driver	<code>boards/zedboard/zed_top_ps.v</code> , <code>vivado/build_ps_system.tcl</code> , <code>sw/</code>	<code>tb_ssc_axi</code> + board

Status: all three testbenches pass, 430 checks in total. The outputs are in [docs/sim_results/](#). The design also synthesises cleanly for 7-series parts: about 2,300 LUTs and 950 flip-flops, roughly 4 % of the Zynq-7020.

Not yet run here: the Vivado scripts and the C program could not be run in the environment where this was written, because it has no Vivado and no ZedBoard. The Verilog was simulated and synthesised with open-source tools, and

the C code was compiled against stand-in Xilinx headers. Your first Vivado run is the real test of the scripts. If a step fails, the manual GUI steps in section 5.4 do the same thing.

1. Folder layout

```
smart_serial_controller/
├─ GETTING_STARTED.md      step-by-step first-day guide
├─ windows/                double-click .bat files: simulate, build Way 1, build Way 2, open the GUI
├─ rtl/                    the controller (synthesisable Verilog)
│   ├─ ssc_sync_filter.v   2-FF synchroniser + glitch filter
│   ├─ ssc_fifo.v          16-entry FIFO
│   ├─ ssc_baud_gen.v       fractional baud-rate generator
│   ├─ ssc_uart.v          UART block
│   ├─ ssc_spi.v           SPI block (master + slave)
│   ├─ ssc_i2c.v           I2C master block
│   ├─ ssc_bridge.v        bridge engine
│   ├─ ssc_irq.v           interrupt controller
│   ├─ ssc_regbank.v       register bank (APB slave) - register map in the header
│   ├─ ssc_apb_top.v       top of the controller (APB)
│   └─ ssc_axi_apb_bridge.v AXI4-Lite -> APB
├─ ssc_axi_top.v           controller as a block-design module (Way 2)
├─ ssc_cmd_fsm.v           tiny APB master for the FPGA-only demo (Way 1)
├─ boards/zedboard/        top levels, pads, pin constraints (XDC)
├─ sim/                    testbenches + fake devices (sim/models)
├─ vivado/                 build_standalone.tcl, build_ps_system.tcl, run_sim.tcl
├─ sw/                     Vitis C program: ssc_regs.h, ssc_driver.c/.h, main.c
└─ docs/                   CODE_WALKTHROUGH.md, REGISTER_MAP.md, sim_results/, pdf/ (all guides as PDF)
```

To understand the code, read [docs/CODE_WALKTHROUGH.md](#). To program the controller, use [docs/REGISTER_MAP.md](#).

2. What you need

Software

Tool	Version	Used for
AMD Vivado ML Standard (free)	2020.2 or newer	Simulation (XSim), synthesis, implementation, bitstream, programming the board (Hardware Manager). The XC7Z020 on the ZedBoard is covered by the free licence.
AMD Vitis (install it together with Vivado)	same version as Vivado	Way 2 only: the C program for the ARM. 2023.2 and newer have the "Vitis Unified IDE"; older versions have "Vitis Classic". Both are described below.
ZedBoard board files	from the Vivado Store	Way 2 only: the DDR3 and MIO settings of the Zynq
Cable drivers	come with Vivado	JTAG programming. On Linux run <code>install_drivers</code> once as root. It is at <code><install>/<ver>/data/xicom/cable_drivers/lin64/install_script/install_drivers/install_drivers</code> for 2025.1 and newer, and at <code><install>/Vivado/<ver>/data/...</code> for older versions.
Windows: JTAG cable driver	comes with Vivado	If Hardware Manager → Auto Connect finds no target: open cmd as Administrator , run <code>cd /d C:\Xilinx\Vivado\<ver>\data\xicom\cable_drivers\nt64</code> (2024.2 and older) or <code>cd /d <install_root>\<ver>\data\xicom\cable_drivers\nt64</code> (2025.1 and newer, where <code>data</code> sits beside <code>Vivado</code>), then run <code>install_drivers_wrapper.bat</code> . If the folder is not there, search the install folder for <code>install_drivers_wrapper.bat</code> . Unplug and replug J17.
Windows: ZedBoard USB-UART (J14) driver	Cypress CY7C64225 USB-UART driver (Avnet ZedBoard support page / Infineon)	Only needed if no new COM port appears in Device Manager → Ports when you plug in J14. The PmodUSBUART's FTDI driver installs itself.
Serial terminal	any	PuTTY or Tera Term (Windows), <code>screen</code> or <code>minicom</code> (Linux), or the serial monitor built into Vitis
<i>Optional:</i> logic-analyser software		Digilent WaveForms (Analog Discovery), Saleae Logic 2 or PulseView. All three have UART, SPI and I2C decoders.
<i>Optional:</i> Icarus Verilog + GTKWave		A free simulator, if you want to run the testbenches without Vivado

Hardware

- ZedBoard (Zynq-7000 XC7Z020) with its 12 V power supply
- 2 micro-USB cables for the ZedBoard: **J17 PROG** (JTAG) and **J14 UART** (the on-board USB-UART for the ARM console)
- **PmodUSBUART** and a micro-USB cable, on **JA**
- **PmodSF3** (32 MB SPI flash), on **JB**
- **PmodTMP2** (ADT7420 temperature sensor), on **JC**, with both address jumpers **open** (address 0x4B)
- *If needed:* 2 × 4.7 kΩ resistors (2.2k to 10k all work) as I2C pull-ups, one from SCL to 3.3 V and one from SDA to 3.3 V.
 - The PmodTMP2 may not have its own pull-ups.

- The FPGA's internal pull-ups usually work at 100 kHz, so try without them first, and add them if `t / temp` reports *No ACK*.

With the Pmod plugged into JC, JC3–JC6 and JC9–JC12 are all filled, so fit the resistors on a breadboard:

1. Run four female-to-male jumper wires from the PmodTMP2's SCL, SDA, GND and VCC pins to four breadboard rows.
2. Run four male-to-male wires from those rows to JC3 (SCL), JC4 (SDA), JC5 (GND) and JC6 (3.3 V).
3. Fit one resistor from SCL to VCC and one from SDA to VCC.

A Digilent Pmod extension cable into a breadboard also works.

- *Optional:* a logic analyser on **JD**. A Digilent PmodTPH2 test-point header makes clipping on easy.

Board jumpers

- **Boot mode = JTAG:** set JP7–JP11 (MIO2–MIO6) all to the GND side.
- **VADJ (J18):** leave it at the default. The constraint file uses LVCMOS25 for the BTNC button. If your J18 is set to 1.8 V, change that one line in `boards/zedboard/zed_standalone.xdc` to LVCMOS18.

Plugging in the Pmods

Pin 1 of every Pmod connector is marked on the board, and pins 1–6 form the top row.

Port	Pmod	Notes
JA	PmodUSBUART	6-pin, top row. JA1 = RTS, JA2 = RXD, JA3 = TXD, JA4 = CTS (the Pmod's own labels). Set the Pmod's power jumper JP1 to LCL , because the ZedBoard powers itself.
JB	PmodSF3	12-pin, fills the whole port. JB1 = CS#, JB2 = MOSI, JB3 = MISO, JB4 = SCK. JB7 is not connected on the PmodSF3 (spare CS1#). JB8 = RST#, JB9 = WP#, JB10 = HOLD#, all held high.
JC	PmodTMP2	Its 2×4 I2C header goes into the four columns nearest the GND/VCC end : pins 3–6 and 9–12, so SCL = JC3 and SDA = JC4
JD	logic analyser (optional)	JD1 UART TX, JD2 UART RX, JD3 SPI SCLK, JD4 MOSI, JD7 MISO, JD8 CS#, JD9 I2C SCL, JD10 SDA. GND is on pins 5 and 11.

If your Pmod revision uses a different pin order, change only the pin lines in `boards/zedboard/zed_pmods.xdc`. The Verilog does not need to change.

3. Simulate first (no board needed)

There are three self-checking testbenches. "Self-checking" means each one compares every result with the expected value and prints PASS or FAIL. You do not have to read waveforms to know whether it works.

Testbench	What it simulates	Sim time	Checks
<code>sim/tb_ssc_top.v</code>	The controller with fake devices: PC terminal, SPI flash, generic SPI slave, outside SPI master, ADT7420 sensor, 10-bit I2C device, second I2C master, I2C bus logger	11.5 ms	411
<code>sim/tb_ssc_axi.v</code>	The Way 2 hardware path: AXI4-Lite handshakes → bridge → controller, with the same register sequences the C driver uses	0.3 ms	9
<code>sim/tb_zed_standalone.v</code>	The complete Way 1 bitstream, with keys typed on the terminal	26 ms	10

Option A: Vivado, one command per testbench

Windows: Vivado's output can exceed Windows' 260-character path limit. Copy the `smart_serial_controller` folder to a short path such as `C:\ssc` first, and run everything from there. Do not build inside `Downloads\...\-main\...\-main\`.

Windows, easiest way: double-click `windows\1_simulate.bat`.

Windows, by hand: use a normal **Command Prompt** (`cmd.exe`), not the Vivado Tcl Shell:

```
call C:\Xilinx\Vivado\2023.2\settings64.bat
cd /d C:\ssc\smart_serial_controller
vivado -mode batch -source vivado/run_sim.tcl -tclargs tb_ssc_top
vivado -mode batch -source vivado/run_sim.tcl -tclargs tb_ssc_axi
vivado -mode batch -source vivado/run_sim.tcl -tclargs tb_zed_standalone
```

Change the first line to your own version and install folder. From 2025.1 the version folder comes before Vivado, for example C:\Xilinx\2025.1\Vivado\settings64.bat or C:\AMDDesignTools\2025.2\Vivado\settings64.bat.

Inside the Vivado GUI Tcl Console or the Vivado Tcl Shell: that prompt already is Vivado, so do not type `vivado` there. Use forward slashes:

```
cd C:/ssc/smart_serial_controller
set argv tb_ssc_axi ; source vivado/run_sim.tcl
```

Linux:

1. source `<Vivado>/settings64.sh`
2. cd to the folder.
3. Run the three `vivado -mode batch ...` lines shown above.

Each run takes 2-5 minutes. The script stops with an error if any check fails.

Option B: Vivado GUI, with waveforms

1. Run the Way 1 build once: double-click `windows\2_build_way1_fpga_only.bat` (see section 4.1). It creates `build/standalone/ssc_standalone.xpr` with all simulation files already added. You can also create a project by hand: add `rtl/*.v`, the two `boards/zedboard/*.v` files as design sources, and `sim/*.v` plus `sim/models/*.v` as simulation sources.
2. In *Sources* → *Simulation Sources* → *sim_1*, right-click the testbench you want and choose **Set as Top**.
3. Click **Flow Navigator** → **Run Simulation** → **Run Behavioral Simulation**.
4. Vivado stops after 1000 ns. Type these two lines in the Tcl Console at the bottom:

```
log_wave -r /
run all
```

The first line records every signal, so anything you drag into the waveform window afterwards still has its full history. The Tcl Console shows the test report. If you already ran without `log_wave`, type `restart` and then the two lines again.

5. Useful signals to drag into the waveform window, from the *Scope* and *Objects* panels:

Test	Signals
UART	<code>dut/u_uart/tx_state</code> , <code>uart_txd</code> , <code>dut/u_uart/u_baud/tick</code>
SPI	<code>spi_sclk</code> , <code>spi_mosi</code> , <code>spi_miso</code> , <code>spi_cs_n</code> , <code>dut/u_spi/m_state</code>
I2C	<code>scl</code> , <code>sda</code> , <code>dut/u_i2c/st</code> , <code>dut/u_i2c/bitn</code> , <code>dut/u_i2c/bus_busy</code>
Bridge	<code>dut/u_bridge/go0</code> , <code>dut/u_bridge/go1</code>

Option C: Icarus Verilog (free, optional)

Linux / macOS:

```
sh sim/run_iverilog.sh
sh sim/run_iverilog.sh vcd
```

The first line runs all three testbenches, which takes about 1-2 minutes. The second line also writes `tb_ssc_top.vcd` in this folder for GTKWave.

Windows:

1. Install Icarus Verilog from bleyer.org/icarus. Tick "Add executable folder(s) to the user PATH".

2. Install Git for Windows.
3. Right-click the `smart_serial_controller` folder and choose **Open Git Bash here**.
4. Type `sh sim/run_iverilog.sh`.

This does not work in cmd.exe or PowerShell. If you only have Vivado, use Option A.

What you should see

The full logs are in `docs/sim_results/`. They were made with Icarus Verilog, so a Vivado run also shows Vivado's own INFO lines around the same testbench output. A shortened example:

```
[2] UART TX: send 'A' (0x41) at 115200 8N1 - the slide 9 example
    line: start 0 | 1 0 0 0 0 0 1 0 | stop 1
[9] Baud-rate accuracy at every standard speed (100 MHz clock)
    baud      divisor      measured bit  error   integer-only divider
    115200    54 + 4/16      8680.00 ns  0.006 %  54 -> 0.47 %
    460800    13 + 9/16      2170.00 ns  0.006 %  14 -> 3.22 %
[11] SPI: read the PmodSF3 JEDEC ID (command 0x9F)
    ID = 20 ba 19
[17] I2C worked example: read the PmodTMP2 temperature (100 kHz)
    bytes 0x0c 0x80 -> 0x0c80 >> 3 = 400 -> 25.00 C
    bus: S 96 A 00 A Sr 97 A 0c A 80 N P          <- exactly slide 13
[23] I2C arbitration: a second master starts at the same moment
...
ALL 411 CHECKS PASSED
```

4. Way 1: FPGA only (quick bring-up)

No ARM and no software are involved. `ssc_cmd_fsm` drives the registers itself, so this is the fastest way to see the hardware work on the board.

4.1 Build the bitstream

- **Windows:** double-click `windows\2_build_way1_fpga_only.bat`.
- **Linux, or a cmd.exe after settings64.bat:** `vivado -mode batch -source vivado/build_standalone.tcl` from the `smart_serial_controller` folder.

This takes 5-15 minutes, depending on the PC. At the end it prints:

```
Way 1 build finished
worst setup slack (WNS) : x.xxx ns    (must be >= 0)
bitstream                : ../build/standalone/zed_top_standalone.bit
```

If WNS is 0 or positive, the design meets 100 MHz. The full report is in `build/standalone/timing_summary.rpt` ("All user specified timing constraints are met"). The resource numbers for your report are in `utilization.rpt`.

You can do the same in the GUI:

1. Create an RTL project for part **xc7z020clg484-1**.
2. Add `rtl/*.v`, `boards/zedboard/zed_top_standalone.v` and `zed_pmod_pads.v`.
3. Add the constraints `zed_pmods.xdc` and `zed_standalone.xdc`.
4. Set `zed_top_standalone` as top.
5. Click **Generate Bitstream**.

4.2 Program the ZedBoard

1. Plug in the Pmods (section 2). Connect J17 (PROG) and the PmodUSBUART's USB cable to the PC, then power on.
2. In Vivado: **Open Hardware Manager → Open Target → Auto Connect → Program Device**. Choose `zed_top_standalone.bit` and click **Program**. The blue DONE LED lights up.
3. Open a terminal on the **PmodUSBUART's** COM port. This is a different port from the ZedBoard's own USB-UART. Use **115200 baud, 8 data bits, no parity, 1 stop bit, no flow control**.
4. Press **BTNC** (the centre button) to reset.

4.3 Expected output

```
Smart Serial Controller - Way 1 bring-up
t = temperature (PmodTMP2 on JC)
f = flash JEDEC ID (PmodSF3 on JB)
> t
Temp = +24.6875 C          <- your room temperature
> f
Flash ID = 20 BA 19        <- Micron 256 Mbit
> x                        <- other keys are echoed
```

What the LEDs show:

LED	Meaning
LD0	Heartbeat
LD1	UART TX activity
LD2	UART RX activity
LD3	SPI activity
LD4	I2C activity
LD5	SPI slave mode
LD6	Last command failed
LD7	Running

In Way 2 (ARM + FPGA), LD0-LD5 mean the same. LD6 lights while the controller's interrupt line is high, and LD7 is on whenever the bitstream is loaded.

To check that the sensor is really measuring, put your finger on it and press `t` again. The temperature should rise.

5. Way 2: ARM + FPGA (the final system)

5.1 Build the hardware

- **Windows:** double-click `windows\3_build_way2_arm_fpga.bat`.
- **Linux, or a cmd.exe after `settings64.bat`:** `vivado -mode batch -source vivado/build_ps_system.tcl`

This takes 10-20 minutes. The script does five things:

1. Creates a block design with the **Zynq PS**, using the ZedBoard preset.
2. Adds the controller as a module (`ssc_axi_top`) and connects it to **M_AXI_GP0** at address **0x43C0_0000**, with its interrupt on **IRQ_F2P[0]** (ID 61).
3. Generates the wrapper and adds `zed_top_ps.v`.
4. Builds the bitstream.
5. Exports `build/ps_system/ssc_system.xsa` for Vitis.

If it stops with *"ZedBoard board files are not installed"*, install them from **Tools → Vivado Store → Boards → Avnet → ZedBoard** and run the script again.

In Vivado 2020.x the store is not under Tools. Instead, go to **File → New Project → ... → Boards** tab, click **Refresh**, then click the download icon next to ZedBoard.

Open `build/ps_system/ssc_ps.xpr` and **Open Block Design** to see the result. It is the architecture slide drawn by Vivado.

About the AXI-APB bridge: the slide shows Xilinx's AXI APB Bridge IP. This project uses a small Verilog bridge instead (`rtl/ssc_axi_apb_bridge.v`). It does the same job, AXI4-Lite in and APB out, and it has two advantages. The whole path from the ARM to the registers can be simulated (`tb_ssc_axi`). And the controller drops into the block design as one module whose address Vivado assigns automatically, with no IP packaging. If your guide wants the Xilinx IP, package `ssc_apb_top` with **Tools → Create and Package New IP**, give its APB interface a memory map of 4 KB, and connect it behind an AXI APB Bridge. The registers and the C code stay the same.

5.2 Create the software in Vitis

The C sources are in `sw/`: `ssc_regs.h`, `ssc_driver.h`, `ssc_driver.c` and `main.c`.

Vitis Unified IDE (2023.2 and newer):

1. Start Vitis and choose an empty workspace folder.
2. **File → New Component → Platform:**
 - Name: `ssc_platform`
 - Hardware design: browse to `build/ps_system/ssc_system.xsa`
 - Operating system: standalone; processor: `ps7_cortexa9_0`
 - Click **Finish**, select the platform in the *Flow* panel, then click **Build**.
3. **File → New Component → Application:** name it `ssc_app`, choose `ssc_platform` and the standalone domain, then click **Finish**.
4. Copy the four files from `sw/` into `ssc_app/src`. You can drag them in, or right-click *Sources* → *Import*.
5. Click **Build** in the *Flow* panel.

Vitis Classic (2020.2 - 2023.1):

1. **File → New → Application Project → Next.**
2. On *Create a new platform from hardware (XSA)*, browse to `ssc_system.xsa`, then click **Next**.
3. Name the application `ssc_app` and select processor `ps7_cortexa9_0`, then click **Next → Next**.
4. Choose the template **Empty Application (C)** and click **Finish**.
5. Right-click `ssc_app/src` → *Import Sources*, choose the `sw/` folder, select all four files, and click **Finish**.
6. Click the hammer icon to build.

5.3 Run it

1. Connect **J14 (UART)** and **J17 (PROG)**. The boot jumpers must be set to JTAG. Power on.
2. Open a terminal on the **ZedBoard's own** USB-UART COM port at 115200 8N1. This is the ARM console. Optionally, open a second terminal on the PmodUSBUART port. That one is the controller's UART.
3. Start the program:
 - **Unified IDE:** click **Run** in the Flow panel.
 - **Classic:** right-click `ssc_app` and choose **Run As → Launch Hardware**.

Vitis programs the bitstream, initialises the PS and starts `main()`.

Expected output on the ARM console:

```

=== Smart Serial Controller - Way 2 (ARM + FPGA) ===
reading the controller ID at 0x43c00000 (a hang here = bitstream not loaded)
controller found at 0x43c00000

Commands:
  selftest      loop-back tests inside the FPGA (no Pmods needed)
  ...           (the full command list)

ssc> selftest                                <- works with no Pmods plugged in
ID register      : 0x53534301 PASS
scratch register : 0xa5a55a5a PASS
UART loop-back   : 64 bytes, 0 errors PASS
SPI loop-back    : 64 words, 0 errors PASS
interrupts seen  : <n>
ssc> temp
raw 0x0c 0x58 -> (0x0c58 >> 3) x 0.0625 = +24.6875 C
ssc> id
JEDEC ID: 20 ba 19 (Micron - PmodSF3)
ssc> flash
erase 4 KB at 0xff0000 : <t> us
program 256 bytes      : <t> us
read back 256 bytes    : 0 errors PASS
first bytes: 5a 5b 58 59 ...
ssc> send hello                                <- "hello" appears in the PmodUSBUART terminal
sent 5 characters
ssc> bridge echo                                <- now type in the PmodUSBUART terminal
UART -> UART bridge on: type in the PmodUSBUART terminal, every key
comes straight back. 'bridge off' to stop.
ssc> bridge off
bridge moved <n> bytes on channel 0, 0 on channel 1 - with no CPU work
ssc> bench
16 bytes at 115200 baud (about 1389 us on the wire):
  CPU time with the 16-entry FIFO : <n> ns
  CPU time byte by byte (no FIFO) : <n> us
  -> the FIFO frees the CPU for about <n> x longer

```

Your exact temperatures and timings will differ. The `bench` command gives the "how much CPU time the FIFOs save" number from slide 12.

If `controller found` does not appear, see the troubleshooting table in section 7.

5.4 Building the block design by hand (if you want to show it in the viva)

1. Create a project with the board **ZedBoard** and add the `rtl/*.v`, `zed_top_ps.v` and `zed_pmod_pads.v` files, plus `zed_pmods.xdc`.
2. **Create Block Design** named `system`. Add **ZYNQ7 Processing System**, then click **Run Block Automation** (apply the board preset).
3. Double-click the Zynq:
 - *Clock Configuration*: FCLK_CLK0 = 100 MHz.
 - *Interrupts*: enable Fabric Interrupts → PL-PS IRQ_F2P.
4. Right-click the canvas and choose **Add Module... → ssc_axi_top**.
 - Select the new block. In the **Block Properties** window, change its **Name** to `ssc_0`, because the steps below use that name.
 - Click **Run Connection Automation** and tick `ssc_0/S_AXI`. This adds the interconnect and reset.
5. Connect `ssc_0/irq` to `IRQ_F2P`.
6. Select each `ssc_0` pin except `S_AXI`, the clock, the reset and `irq`, and press **Ctrl+T** (Make External).
 - Vivado names each new port `<pin>_0`. Click each port and, in *External Port Properties*, delete the `_0`, so the port has exactly the pin name.
 - The full list is `uart_rxd`, `uart_txd`, `uart_cts_n`, `uart_rts_n`, `spi_sclk`, `spi_mosi`, `spi_miso`, `spi_cs_n` (4 bits), `spis_sclk`, `spis_mosi`, `spis_cs_n`, `spis_miso`, `spis_miso_oe`, `spi_slave_mode`, `i2c_scl_in`, `i2c_scl_oe`, `i2c_sda_in` and `i2c_sda_oe`.
 - These names must match the `system_wrapper u_system (...)` instance in `zed_top_ps.v`, not that file's own top-level ports.
 - Also make **FCLK_CLK0** external and rename it `fclk`, and make `ssc_0/irq` external and rename it `irq_out`.

7. In the **Address Editor**, set `ssc_0` to `0x43C0_0000` with a range of 4K.
8. Validate the design (F6), then **Create HDL Wrapper** and choose "let Vivado manage". Set `zed_top_ps` as top.
9. **Generate Bitstream**, then **File → Export → Export Hardware** with "include bitstream".

6. How to verify the output: checklist for the report

Level 1: simulation, automatic

- All three testbenches end with `ALL n CHECKS PASSED`.
- Take screenshots of the XSim waveforms for:
 - one UART frame ('A')
 - one SPI flash ID transfer
 - the I2C temperature read, with S, Sr, ACK, NACK and P marked
 - the arbitration test
 - clock stretching (SCL held low by the sensor)

Level 2: Way 1 on the board

- The banner appears after BTNC is pressed.
- `t` gives a sensible room temperature, and it rises when you touch the sensor.
- `f` prints `20 BA 19`.
- With the PmodTMP2 unplugged, `t` prints `No ACK...` and LD6 lights.

Level 3: Way 2 on the board

- `selftest` passes with no Pmods attached.
- `temp`, `id`, `flash`, `send`, `rx`, `bridge echo` and `bench` behave as shown in section 5.3.
- `regs` shows the interrupt count growing, which proves the IRQ path works.

Level 4: logic analyser on JD (real waveforms)

Set up the decoders:

Decoder	Channel(s)	Settings
UART	JD1	115200 8N1
SPI	CLK JD3, MOSI JD4, MISO JD7, CS JD8	mode 0, MSB first
I2C	SCL JD9, SDA JD10	

Ground goes to JD pin 5 or 11. What you should capture:

Action	What the capture shows
Press <code>t</code>	Start, Write <code>0x4B</code> , ACK, <code>0x00</code> , ACK, Repeated Start, Read <code>0x4B</code> , ACK, <code>0x0C</code> , ACK, <code>0x80</code> , NACK, Stop (the data bytes are your real temperature)
Press <code>f</code>	CS# low, MOSI <code>9F 00 00 00</code> , MISO <code>xx 20 BA 19</code> , CS# high
Any character	One UART frame with start, data LSB first, and stop. Measure the bit width: 8.68 μ s at 115200.

Compare the I2C capture with the strip on slide 13.

Optional: Integrated Logic Analyzer (ILA) inside the FPGA

1. Before synthesis, open **Settings → Synthesis** and set **-fsm_extraction** to **none**. This keeps the I2C state numbers the same as in `ssc_i2c.v`. Then run synthesis.
2. Open **Synthesized Design → Set Up Debug**. Using **Find Nets to Add**, pick `u_ssc/u_i2c/st[3:0]`, `c_scl_oe`, `c_sda_oe` and `u_ssc/u_i2c/sh[7:0]`. Accept the defaults (the clock is the 100 MHz clock).
3. Implement and generate the bitstream. Then click **Program Device** and keep the paths Vivado fills in: `build\standalone\ssc_standalone.runs\impl_1\zed_top_standalone.bit`, and the `.ltx` probes file beside it.

Do not browse to `build\standalone\zed_top_standalone.bit`. That file is a copy made only by `2_build_way1_fpga_only.bat`, and it has no ILA. The same applies whenever you rebuild in the GUI after changing the Verilog.

4. In Hardware Manager, set the trigger `st == 3` (S_ST2 = START), arm it, and press `t` in the terminal.

Numbers for the results chapter

Number	Where to find it
Resource use	<code>utilization.rpt</code> (expect about 2-3 k LUTs, 1 k FFs, a few LUTRAMs)
Timing	WNS from <code>timing_summary.rpt</code>
Baud error table	test [9] in the simulation log
I2C SCL frequency	test [18], about 383 kHz at prescale 62
CPU time with and without the FIFO	the <code>bench</code> command
Bytes moved by the bridge with zero CPU instructions	<code>bridge off / BRIDGE_CNT</code>

7. Troubleshooting

Symptom	Fix
Simulation stops at 1000 ns in the GUI	Type <code>log_wave -r /</code> and then <code>run all</code> in the Tcl console
A <code>.bat</code> window says "Could not find Vivado"	Find <code>settings64.bat</code> in your install folder. It is at <code><root>\<version>\Vivado\settings64.bat</code> for 2025.1 and newer, and <code>C:\Xilinx\Vivado\<version>\settings64.bat</code> for older versions. Then edit line 6 of <code>windows_find_vivado.bat</code> to read, for example, <code>set "MY_VIVADO_SETTINGS=C:\Xilinx\2025.1\Vivado\settings64.bat"</code> , with one pair of quotes around the whole line.
Strange "file not found" errors during a Windows build	The path is too long. Copy the folder to <code>C:\ssc\smart_serial_controller</code> and build there.
No banner in Way 1	Use the PmodUSBUART's COM port, not the ZedBoard's. Press BTNC. Check the Pmod is in the top row of JA. If the Pmod revision has TXD/RXD swapped, swap the <code>uart_txd / uart_rxd</code> pins in <code>zed_pmods.xdc</code> .
Garbled characters	The terminal must be set to 115200 8N1, flow control off
No ACK from PmodTMP2	The PmodTMP2 must sit in JC pins 3-6/9-12, with both jumpers open (0x4B). With jumpers fitted the address is 0x48-0x4A: change <code>TMP2_ADDR</code> in <code>ssc_cmd_fsm.v</code> or <code>ssc_driver.h</code> . If it still fails, add the SCL/SDA pull-up resistors on a breadboard (section 2, Hardware).
Flash ID FF FF FF or 00 00 00	Check that the PmodSF3 is in JB the right way round (pin 1 to pin 1)
Way 2 prints reading the controller ID ... and then nothing	The CPU is stuck on its first read of the controller because the FPGA is not programmed. Tick "Program FPGA" / "Program Device" in the run configuration, or program the bitstream from the Vivado Hardware Manager first. Then run again.
ERROR: ID register reads 0x... (Way 2)	The FPGA answers, but it holds a different design or the address is wrong. Check the Address Editor against <code>SSC_BASEADDR</code> in <code>sw/ssc_regs.h</code> , and check that the bitstream is the one from this project.
No COM port for J14 (Way 2 console)	Install the Cypress CY7C64225 USB-UART driver (section 2) and replug J14. To tell the two COM ports apart, unplug one cable and see which COMx disappears.
Hardware Manager: no hardware target	Check you used J17 (PROG), not J14. Re-install the cable driver as described in section 2.
Vitis: <code>xtime_l.h</code> or <code>xiltimer.h</code> not found	<code>sw/</code> picks the right header automatically: <code>xiltimer.h</code> on 2023.2+ and <code>xtime_l.h</code> on older versions. Make sure you created a <i>standalone</i> platform for <code>ps7_cortexa9_0</code> .
Vitis: <code>XPAR_XSCUGIC_0_BASEADDR</code> or <code>XPAR_SCUGIC_SINGLE_DEVICE_ID</code> undeclared	<code>ssc_driver.c</code> picks whichever one your Vitis defines. Make sure the BSP includes the <code>scugic</code> driver (it does by default).
Vivado: ZedBoard board part not found	Tools → Vivado Store → Boards → ZedBoard → Install. In 2020.x, use the Boards tab of the New Project wizard.
Vivado: <code>BITSTREAM.CONFIG.UNUSEDPIN</code> error	Delete that last line of <code>zed_pmods.xdc</code> . It only matters for the duplicated SCL/SDA pins of the PmodTMP2.
<code>apply_bd_automation</code> error in <code>build_ps_system.tcl</code>	Open <code>build/ps_system/ssc_ps.xpr</code> in the GUI, delete the block design <code>system</code> if it was created, and do all of section 5.4 by hand. A script that stopped with an error cannot be resumed part-way.
Timing not met (negative WNS)	Should not happen at 100 MHz. Check that only the GCLK / FCLK0 clock is used, and open <code>timing_summary.rpt</code> to see the failing path.